

تمرين تطبيقي (مختبر الجزء العاشر)

1. استغلال SSRF (Server-Side Request Forgery)

SSRF: (أ) مثال على ثغرة

php

<?php

// × بدون حماية URL خطأ: خدمة جلب

```
if (isset($_GET['url'])) {  
    $url = $_GET['url'];
```

// خارجي URL جلب محتوى من

```
$content = file_get_contents($url);
```

// أو استخدام cURL

```
$ch = curl_init();
```

```
curl_setopt($ch, CURLOPT_URL, $url);
```

```
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
```

```
$response = curl_exec($ch);
```

```
curl_close($ch);
```

```
echo $response;
```

```
}
```

?>

<!-- مثال على استغلال

<!-- جلب ملفات داخلية

http://example.com/fetch.php?url=http://localhost:8080/admin

http://example.com/fetch.php?url=file:///etc/passwd

http://example.com/fetch.php?url=http://169.254.169.254/latest/meta-data/

<!-- استغلال خدمات داخلية

http://example.com/fetch.php?url=http://127.0.0.1:3306

http://example.com/fetch.php?url=http://internal-api.local

Redis: (ب) استغلال

php

<?php

// × خطأ: يسمح بالوصول لخدمات داخلية

```
if (isset($_GET['url'])) {
```

```

$url = $_GET['url'];

// قد يستطيع المهاجم الوصول لـ Redis
// مثال: http://target.com/fetch.php?url=http://127.0.0.1:6379
$content = @file_get_contents($url);

// أو حقن أوامر في Redis
//
http://target.com/fetch.php?url=http://127.0.0.1:6379/FLUSHALL
echo htmlspecialchars($content);
}
?>

```

2. Whitelist باستخدام SSRF منع

(أ) الحماية الأساسية

```

php
<?php
class SSRFProtection {
    private $allowed_domains = [
        'api.trusted.com',
        'cdn.example.com',
        'images.safe-site.net'
    ];

    private $allowed_schemes = ['http', 'https'];
    private $blocked_ips = [
        '127.0.0.0/8',
        '10.0.0.0/8',
        '172.16.0.0/12',
        '192.168.0.0/16',
        '169.254.0.0/16',
        '0.0.0.0/8'
    ];

    public function fetchURL($url) {
        // 1. التحقق من صحة URL
        if (!$this->isValidURL($url)) {
            throw new Exception('رابط غير صالح');
        }
    }
}

```

```

// 2. التحقق من الـ Whitelist
if (!$this->isDomainAllowed($url)) {
    throw new Exception('النطاق غير مسموح به');
}

// 3. التحقق من عدم الوصول لشبكات داخلية
if ($this->isInternalNetwork($url)) {
    throw new Exception('غير مسموح بالوصول للشبكات الداخلية');
}

// 4. مع إعدادات آمنة cURL استخدام
return $this->safeCurlFetch($url);
}

private function isValidURL($url) {
    $parsed = parse_url($url);

    if (!$parsed || !isset($parsed['host'])) {
        return false;
    }

    // HTTP/HTTPS السماح فقط بـ
    if (!in_array($parsed['scheme'] ?? '', $this->allowed_schemes)) {
        return false;
    }

    return true;
}

private function isDomainAllowed($url) {
    $host = parse_url($url, PHP_URL_HOST);

    foreach ($this->allowed_domains as $allowed) {
        if ($host === $allowed || fnmatch($allowed, $host)) {
            return true;
        }
    }
}

```

```

        return false;
    }

    private function isInternalNetwork($url) {
        $host = parse_url($url, PHP_URL_HOST);
        $ip = gethostbyname($host);

        foreach ($this->blocked_ips as $blocked) {
            if ($this->ipInRange($ip, $blocked)) {
                return true;
            }
        }

        return false;
    }

    private function ipInRange($ip, $range) {
        if (strpos($range, '/') === false) {
            $range .= '/32';
        }

        list($subnet, $bits) = explode('/', $range);
        $ip = ip2long($ip);
        $subnet = ip2long($subnet);
        $mask = -1 << (32 - $bits);
        $subnet &= $mask;

        return ($ip & $mask) == $subnet;
    }

    private function safeCurlFetch($url) {
        $ch = curl_init();

        curl_setopt_array($ch, [
            CURLOPT_URL => $url,
            CURLOPT_RETURNTRANSFER => true,
            CURLOPT_TIMEOUT => 10,
            CURLOPT_CONNECTTIMEOUT => 5,
            CURLOPT_MAXREDIRS => 2,

```

```

        CURLOPT_FOLLOWLOCATION => false, // ⚠ إيقاف Follow
Location لمنع Open Redirect
        CURLOPT_PROTOCOLS => CURLPROTO_HTTP |
CURLPROTO_HTTPS,
        CURLOPT_SSL_VERIFYPEER => true,
        CURLOPT_SSL_VERIFYHOST => 2,
        CURLOPT_HTTPHEADER => [
            'User-Agent: SafeFetcher/1.0'
        ]
    ]);

$response = curl_exec($ch);

    if (curl_errno($ch)) {
        throw new Exception('خطأ في جلب البيانات: ' .
curl_error($ch));
    }

    curl_close($ch);
    return $response;
}
}

```

// الاستخدام الآمن

```

$protector = new SSRFProtection();
try {
    $content = $protector->fetchURL($_GET['url']);
    echo $content;
} catch (Exception $e) {
    echo "خطأ: " . $e->getMessage();
}
?>

```

3. تنفيذ Open Redirect

Open Redirect: (أ) مثال على ثغرة

php

```
<?php
```

// ✗ Open Redirect خطأ: صفحة تسجيل دخول مع

```
session_start();
```

```
// بعد تسجيل الدخول الناجح
if ($login_successful) {
    $redirect_url = $_GET['redirect'] ?? '/dashboard.php';

    // ✗ خطأ: التحويل بدون تحقق
    header("Location: $redirect_url");
    exit;
}

// ✗ Open Redirect خطأ: صفحة تسجيل خروج مع
if (isset($_GET['logout'])) {
    session_destroy();
    $next = $_GET['next'] ?? '/login.php';
    header("Location: $next");
    exit;
}
?>

<!-- أمثلة على استغلال -->
<!-- التصيد: إرسال رابط للمستخدم -->
http://example.com/login.php?redirect=http://evil.com/phishing
http://example.com/logout.php?next=http://evil.com/steal-cookies

<!-- اختراق OAuth -->
http://example.com/oauth/callback?code=xyz&redirect=https://attacker.com

4. إصلاح Open Redirect
الحماية باستخدام Whitelist:
php
<?php
class RedirectProtection {
    private $allowed_redirects = [
        '/dashboard.php',
        '/profile.php',
        '/home.php',
        '/',
        '/settings.php'
    ];
};
```

```

private $allowed_external = [
    'https://trusted-partner.com',
    'https://oauth.google.com',
    'https://api.example.com/callback'
];

public function safeRedirect($url) {
    // إذا كان رابط داخلي نسبي
    if (strpos($url, '/') === 0) {
        return $this->validateInternalRedirect($url);
    }

    // إذا كان رابط خارجي كامل
    return $this->validateExternalRedirect($url);
}

private function validateInternalRedirect($url) {
    // التحقق من المسارات المسموح بها
    foreach ($this->allowed_redirects as $allowed) {
        if ($url === $allowed) {
            return $url;
        }
    }

    // الافتراضي الآمن
    return '/dashboard.php';
}

private function validateExternalRedirect($url) {
    $parsed = parse_url($url);

    if (!isset($parsed['host'])) {
        return '/dashboard.php';
    }

    // الخارجي Whitelist التحقق من الـ
    foreach ($this->allowed_external as $allowed) {
        $parsed_allowed = parse_url($allowed);
    }
}

```

```

        if ($parsed['host'] === $parsed_allowed['host']) {
            // التحقق من البروتوكول
            if (($parsed['scheme'] ?? 'https') ===
($parsed_allowed['scheme'] ?? 'https')) {
                return $url;
            }
        }
    }

    return '/dashboard.php';
}

// طريقة بديلة: استخدام التوكنات
public function generateRedirectToken($url) {
    $secret = 'your-secret-key';
    $timestamp = time();
    $token = hash_hmac('sha256', $url . $timestamp, $secret);

    $_SESSION['redirect_token'] = $token;
    $_SESSION['redirect_url'] = $url;
    $_SESSION['redirect_expiry'] = $timestamp + 300; // دقائق 5

    return $token;
}

public function validateRedirectToken($token, $url) {
    if (!isset($_SESSION['redirect_token']) ||
        !isset($_SESSION['redirect_url']) ||
        !isset($_SESSION['redirect_expiry'])) {
        return false;
    }

    if (time() > $_SESSION['redirect_expiry']) {
        return false;
    }

    if (!hash_equals($_SESSION['redirect_token'], $token)) {
        return false;
    }
}

```

```

        if ($_SESSION['redirect_url'] !== $url) {
            return false;
        }

        return true;
    }
}

// الاستخدام في تسجيل الدخول
$redirect = new RedirectProtection();

// إنشاء رابط تسجيل الدخول
if ($login_successful) {
    $requested_url = $_GET['redirect'] ?? '/dashboard.php';
    $safe_url = $redirect->safeRedirect($requested_url);

    header("Location: $safe_url");
    exit;
}

// أو مع التوكنات
if ($login_successful) {
    if (isset($_GET['redirect']) && isset($_GET['token'])) {
        if ($redirect->validateRedirectToken($_GET['token'],
$_GET['redirect'])) {
            header("Location: {$_GET['redirect']}");
            exit;
        }
    }

    header("Location: /dashboard.php");
    exit;
}
?>

```

5. استغلال BOLA (Broken Object Level Authorization)

REST API في BOLA (أ) مثال على ثغرة

```

php
<?php

```

```

// ✗ خطأ: API الملكية بدون تحقق
class UserAPI {
    public function getOrders($user_id, $order_id) {
        // استعلام قاعدة البيانات
        $stmt = $conn->prepare("SELECT * FROM orders WHERE id = ?");

        $stmt->bind_param("i", $order_id);
        $stmt->execute();
        $result = $stmt->get_result();
        $order = $result->fetch_assoc();

        // ✗ خطأ: إرجاع الطلب بدون التحقق من ملكية المستخدم
        return $order;
    }

    public function updateProfile($user_id, $profile_data) {
        // ✗ خطأ: user_id في المسار user_id بدون تحقق من
        $profile_id = $_GET['profile_id'] ?? $user_id;

        $stmt = $conn->prepare("UPDATE profiles SET data = ? WHERE id = ?");
        $stmt->bind_param("si", json_encode($profile_data), $profile_id);
        $stmt->execute();

        return ['success' => true];
    }
}

// استغلال BOLA
// المستخدم رقم 1 يصل لبيانات المستخدم رقم 2
GET /api/user/1/orders/100 # طلب المستخدم 1
GET /api/user/1/orders/200 # طلب المستخدم 2 - يجب منعه

// أو تغيير المعامل في الجسم
PUT /api/user/1/profile
{
    "user_id": 2, // ⚠ يحاول تعديل حساب شخص آخر
    "email": "hacker@evil.com"
}

```

```
}  
?>
```

6. تطبيق تحقق الملكية (Object Level Authorization)

BOLA: الحماية من

php

```
<?php
```

```
class SecureUserAPI {  
    private $conn;
```

```
    public function __construct($conn) {  
        $this->conn = $conn;  
    }
```

```
    public function getOrders($requested_user_id, $order_id,  
$authenticated_user_id) {
```

```
        // 1. التحقق من أن المستخدم يطلب بياناته فقط
```

```
        if ((int)$requested_user_id !==  
(int)$authenticated_user_id) {  
            http_response_code(403);  
            return ['error' => 'غير مصرح بالوصول'];  
        }
```

```
        // 2. التحقق من ملكية الطلب
```

```
$stmt = $this->conn->prepare("  
        SELECT o.*  
        FROM orders o  
        JOIN users u ON o.user_id = u.id  
        WHERE o.id = ? AND u.id = ?  
    ");  
$stmt->bind_param("ii", $order_id,  
$authenticated_user_id);  
$stmt->execute();  
$result = $stmt->get_result();  
  
    if ($result->num_rows === 0) {  
        http_response_code(404);  
        return ['error' => 'الطلب غير موجود'];  
    }
```

```

        return $result->fetch_assoc();
    }

    public function updateProfile($profile_id, $profile_data,
    $authenticated_user_id) {
        // 1. مرسل في البيانات user_id تجاهل أي
        unset($profile_data['user_id']);
        unset($profile_data['id']);

        // 2. يعود للمستخدم profile_id التحقق من أن
        $stmt = $this->conn->prepare("SELECT id FROM profiles
WHERE id = ? AND user_id = ?");
        $stmt->bind_param("ii", $profile_id,
    $authenticated_user_id);
        $stmt->execute();
        $result = $stmt->get_result();

        if ($result->num_rows === 0) {
            http_response_code(403);
            return ['error' => 'غير مصرح بالتعديل'];
        }

        // 3. التنفيذ
        $stmt = $this->conn->prepare("UPDATE profiles SET data =
?, updated_at = NOW() WHERE id = ?");
        $stmt->bind_param("si", json_encode($profile_data),
    $profile_id);
        $stmt->execute();

        return ['success' => true];
    }

    // للتحقق Middleware طريقة أفضل: استخدام
    public function checkOwnership($user_id, $resource_type,
    $resource_id) {
        $queries = [
            'order' => "SELECT id FROM orders WHERE id = ? AND
user_id = ?",

```

```

        'profile' => "SELECT id FROM profiles WHERE id = ?
AND user_id = ?",
        'address' => "SELECT id FROM addresses WHERE id = ?
AND user_id = ?"
    ];

    if (!isset($queries[$resource_type])) {
        return false;
    }

    $stmt = $this->conn->prepare($queries[$resource_type]);
    $stmt->bind_param("ii", $resource_id, $user_id);
    $stmt->execute();
    $result = $stmt->get_result();

    return $result->num_rows > 0;
}
}

// Middleware للتحقق من الملكية
class OwnershipMiddleware {
    public function handle($request, $next) {
        $user_id = $request->getAttribute('user_id');
        $resource_id = $request->getAttribute('resource_id');
        $resource_type = $request->getAttribute('resource_type');

        $api = new SecureUserAPI($this->conn);

        if (!$api->checkOwnership($user_id, $resource_type,
$resource_id)) {
            return $response->withStatus(403)->withJson([
                'error' => 'غير مصرح بالوصول لهذا المورد'
            ]);
        }

        return $next($request);
    }
}
}

```

```
// تسلسلية IDs عشوائية بدلاً من UUIDs استخدام
class SecureResource {
    public static function generateSecureId() {
        return bin2hex(random_bytes(16)); // 32 حرف عشوائي
    }

    public function createOrder($user_id, $order_data) {
        $order_id = self::generateSecureId();

        $stmt = $this->conn->prepare("
            INSERT INTO orders (secure_id, user_id, data)
            VALUES (?, ?, ?)
        ");
        $stmt->bind_param("sis", $order_id, $user_id,
            json_encode($order_data));
        $stmt->execute();

        return $order_id;
    }

    public function getOrderById($secure_id, $user_id) {
        $stmt = $this->conn->prepare("
            SELECT * FROM orders
            WHERE secure_id = ? AND user_id = ?
        ");
        $stmt->bind_param("si", $secure_id, $user_id);
        $stmt->execute();

        return $stmt->get_result()->fetch_assoc();
    }
}
?>
```

7. إضافة Rate Limiting

Rate Limiting: تطبيق

php

<?php

```
class RateLimiter {
    private $limits = [
```

```

        'login' => ['attempts' => 5, 'window' => 900], // 5
        محاولات في 15 دقيقة
        'api' => ['requests' => 100, 'window' => 3600], // 100
        طلب في الساعة
        'password_reset' => ['attempts' => 3, 'window' => 3600]
    ];

    private $storage;

    public function __construct($storage = 'session') {
        $this->storage = $storage;
        if ($storage === 'session' && session_status() ===
PHP_SESSION_NONE) {
            session_start();
        }
    }

    public function check($action, $identifier) {
        $limit = $this->limits[$action] ?? $this->limits['api'];
        $key = "rate_limit_{ $action }_{ $identifier }";

        $data = $this->getData($key);

        // إذا كانت البيانات قديمة، إعادة التعيين
        if ($data && time() - $data['timestamp'] >
$limit['window']) {
            $this->reset($action, $identifier);
            $data = null;
        }

        if (!$data) {
            $data = [
                'count' => 1,
                'timestamp' => time(),
                'blocked_until' => null
            ];
            $this->setData($key, $data);
            return true;
        }
    }

```

```

        // التحقق إذا كان محظوراً
        if ($data['blocked_until'] && time() <
$data['blocked_until']) {
            $remaining = $data['blocked_until'] - time();
            throw new Exception("ثانية $remaining محاولات كثيرة. حاول بعد");
        }

        // زيادة العداد
        $data['count']++;

        // إذا تجاوز الحد، حظر
        if ($data['count'] > $limit['attempts']) {
            $data['blocked_until'] = time() + $limit['window'];
            $this->setData($key, $data);

            throw new Exception("تجاوزت الحد المسموح. حاول لاحقاً");
        }

        $this->setData($key, $data);
        return true;
    }

    public function getRemaining($action, $identifier) {
        $limit = $this->limits[$action] ?? $this->limits['api'];
        $key = "rate_limit_{ $action }_{ $identifier }";

        $data = $this->getData($key);

        if (!$data || time() - $data['timestamp'] >
$limit['window']) {
            return $limit['attempts'];
        }

        return max(0, $limit['attempts'] - $data['count']);
    }

    private function getData($key) {
        if ($this->storage === 'session') {

```

```

        return $_SESSION[$key] ?? null;
    }

    // أو قاعدة البيانات هنا Redis يمكن استخدام
    // return $this->redis->get($key);
}

private function setData($key, $data) {
    if ($this->storage === 'session') {
        $_SESSION[$key] = $data;
        return;
    }

    // في Redis: $this->redis->setex($key, 3600,
    json_encode($data));
}

public function reset($action, $identifier) {
    $key = "rate_limit_{ $action }_{ $identifier }";

    if ($this->storage === 'session') {
        unset($_SESSION[$key]);
    }
}

}

// الاستخدام في تسجيل الدخول
$limiter = new RateLimiter();

try {
    $ip = $_SERVER['REMOTE_ADDR'];
    $limiter->check('login', $ip . '_' . $_POST['email']);

    // عملية تسجيل الدخول
    if (verifyLogin($_POST['email'], $_POST['password'])) {
        $limiter->reset('login', $ip . '_' . $_POST['email']);
        // تسجيل الدخول الناجح
    } else {
        throw new Exception('بيانات الدخول غير صحيحة');
    }
}

```

```

    }
} catch (Exception $e) {
    echo "خطأ: " . $e->getMessage();
}

// الاستخدام في API
class RateLimitedAPI {
    private $limiter;

    public function __construct() {
        $this->limiter = new RateLimiter('redis'); // استخدام Redis
        للتخزين
    }

    public function handleRequest($request) {
        $api_key = $request->getHeader('API-Key')[0] ??
        'anonymous';
        $endpoint = $request->getUri()->getPath();

        try {
            $this->limiter->check('api_' . $endpoint, $api_key);

            // معالجة الطلب
            return $this->processRequest($request);
        } catch (Exception $e) {
            // إرجاع هيدر Rate Limit
            header('X-RateLimit-Limit: ' . $this->limiter->getLimit('api'));
            header('X-RateLimit-Remaining: ' . $this->limiter->getRemaining('api_' . $endpoint, $api_key));
            header('X-RateLimit-Reset: ' . (time() + 3600));

            http_response_code(429);
            return ['error' => 'Too Many Requests', 'message' =>
            $e->getMessage()];
        }
    }
}

```

// الموزع Rate Limiting للـ Redis تكامل مع

```
class RedisRateLimiter {
    private $redis;

    public function __construct($redis) {
        $this->redis = $redis;
    }

    public function checkRateLimit($key, $limit, $window) {
        $current = time();
        $redis_key = "ratelimit:$key:$window";

        // لأداء أفضل Redis Pipeline استخدام
        $pipe = $this->redis->pipeline();
        $pipe->zremrangebyscore($redis_key, 0, $current -
$window);
        $pipe->zcard($redis_key);
        $pipe->zadd($redis_key, $current, $current . ':' .
uniqid());
        $pipe->expire($redis_key, $window);
        $result = $pipe->execute();

        $count = $result[1];

        if ($count > $limit) {
            return false;
        }

        return $limit - $count;
    }
}
```

// استخدام في نظام كبير

```
class AdvancedRateLimiter {
    private $limits = [
        'ip' => [
            'default' => ['requests' => 1000, 'window' => 3600],
            'strict' => ['requests' => 100, 'window' => 900]
        ],
    ],
```

```

        'user' => [
            'free' => ['requests' => 100, 'window' => 3600],
            'premium' => ['requests' => 10000, 'window' => 3600]
        ],
        'endpoint' => [
            '/api/login' => ['requests' => 5, 'window' => 900],
            '/api/register' => ['requests' => 3, 'window' =>
3600]
        ]
    ];

    public function check($request) {
        $ip = $request->getClientIp();
        $user_id = $request->getAttribute('user_id');
        $endpoint = $request->getUri()->getPath();

        // تطبيق عدة طبقات من Rate Limiting
        $checks = [
            $this->checkIPLimit($ip),
            $this->checkUserLimit($user_id),
            $this->checkEndpointLimit($endpoint, $ip)
        ];

        return !in_array(false, $checks);
    }

    private function checkIPLimit($ip) {
        // VPN مشترك مثل IP يمكن أن يكون IP حساب مصداقية
        $ip_reputation = $this->getIPReputation($ip);

        if ($ip_reputation === 'bad') {
            $limit = $this->limits['ip']['strict'];
        } else {
            $limit = $this->limits['ip']['default'];
        }

        return $this->incrementAndCheck("ip:$ip", $limit);
    }

```

```

private function checkUserLimit($user_id) {
    if (!$user_id) {
        return true; // مستخدم غير مسجل له حد مختلف
    }

    $user_plan = $this->getUserPlan($user_id);
    $limit = $this->limits['user'][$user_plan] ?? $this->limits['user']['free'];

    return $this->incrementAndCheck("user:$user_id", $limit);
}

private function checkEndpointLimit($endpoint, $ip) {
    if (isset($this->limits['endpoint'][$endpoint])) {
        $limit = $this->limits['endpoint'][$endpoint];
        return $this->incrementAndCheck("endpoint:$endpoint:$ip", $limit);
    }

    return true;
}
}
?>

```

الخلاصة:

أفضل الممارسات المختصرة:

SSRF Protection:

للنطاقات المسموح بها Whitelist استخدم

(127.0.0.1, 10.x.x.x, etc.) امنع الوصول للشبكات الداخلية

قبل جلبها URLs تحقق من الـ

Open Redirect Protection:

للمسارات المسموح بها Whitelist استخدم

Whitelist لا تسمح بروابط خارجية إلا إذا كانت في

استخدم توكنات مؤقتة للتحويلات

BOLA Protection:

تحقق دائماً من ملكية المورد

للتحقق التلقائي Middleware استخدم

تسلسلية IDs عشوائية بدلاً من UUIDs فكر في استخدام

Rate Limiting:

على جميع نقاط الدخول الحرجة Rate Limiting طبق

للأنظمة الكبيرة (Redis) استخدم تخزين موزع

في الاستجابات Rate Limit أضف هيدرات

كود نهائي موحد:

```
php
<?php
// SecureSystem.php - نظام آمن شامل
class SecureSystem {
    private $rateLimiter;
    private $ssrfProtector;

    public function __construct() {
        $this->rateLimiter = new RateLimiter();
        $this->ssrfProtector = new SSRFProtection();
    }

    public function handleAPIRequest($request) {
        // Rate Limiting
        $this->rateLimiter->check('api', $request->getClientIp());

        // تحقق من الملكية (BOLA Protection)
        $this->verifyOwnership($request);
    }
}
```

```
// معالجة الطلب
return $this->processRequest($request);
}

public function safeFetchURL($url) {
    return $this->ssrfProtector->fetchURL($url);
}

public function safeRedirect($url) {
    $validator = new RedirectProtection();
    return $validator->safeRedirect($url);
}
}
?>
```